Implementation Report: Ifpack2-Based Camellia Additive Schwarz Recovery

Nathan V. Roberts / AI-assisted implementation summary

May 28, 2026

1 Purpose

This report summarizes the implementation work carried out to recover the Ifpack2/Tpetra-based `CAMELLIA_ADDITIVE_SCHWARZ` path, with particular focus on the mathematical and operator-level changes in `GMGOperator`.

The original goal of the branch was to replace the older Epetra/Ifpack-based Camellia additive Schwarz path with an Ifpack2/Tpetra-native implementation while preserving the core Camellia semantics:

- one Schwarz subdomain per owned cell,
- each subdomain equal to the owned cell plus its overlap halo,
- support for Camellia-defined overlap in terms of mesh neighbors rather than only rank-based overlap levels.

2 Important Dependency Note

A practical dependency of this work is that the currently used `Ifpack2::OverlappingRowMatrix` is not yet in main Trilinos. The implementation currently relies on Nathan V. Roberts' Trilinos fork:

- repository: `https://github.com/CamelliaDPG/Trilinos.git`
- branch: `fixes-for-camellia`

This does not block merge into the target Camellia branch, because the target branch already depends on changes from the same Trilinos fork/branch.

3 High-Level Outcome

The recovery effort produced several important positive results, but the full production configuration is not yet completely settled across the entire test space.

The most important successful outcomes so far are:

- the outer Ifpack2 overlap+user-partitioner mechanism was validated in focused custom tests, both in serial and on 2 MPI ranks,

- a direct unit test was added and passed showing that the produced per-cell Schwarz parts match the intended overlap-halo dof sets for the tested case,
- duplicated use of the same overlapping user partition inside the inner preconditioner was shown to be harmful,
- the old unimplemented diagonal-weighting exception path in `GMGOperator` was replaced with a working multiplicity-based implementation.

However, the branch is not yet in a completely final state. In particular:

- some problem-level additive Schwarz tests improved dramatically under one smoother configuration,
- but `GMGSolver` multirank identity tests still reveal unresolved inner-solver issues,
- and the best current inner-solver choice appears to differ between the problem-level preconditioning tests and some small multirank GMG identity cases.

So the decisive implementation lesson was real, but the production inner-solver policy is not yet fully stabilized.

4 What Changed in GMGOperator: Conceptual View

The committed branch originally attempted to realize Camellia additive Schwarz by combining:

1. an outer Ifpack2 Schwarz operator with custom overlap rows and custom per-cell parts, and
2. an inner Block Relaxation preconditioner that was also given the same custom user partition.

The key recovery insight was that this duplicated the overlapping partition structure in a way that changed the effective local subdomain solve and degraded the mathematical behavior of the smoother.

After recovery, the intended structure is:

- the *outer* Schwarz layer receives the Camellia per-cell overlapping partition information,
- the *inner* Block Relaxation preconditioner does *not* receive the same user partition a second time.

This keeps the outer Schwarz layer responsible for the cell-centered overlapping decomposition, while avoiding an additional nested repartitioning of the same local problem.

5 Mathematical Structure Before and After

5.1 Target Schwarz semantics

The desired additive Schwarz preconditioner is conceptually of the form

$$M^{-1} = \sum_{i=1}^{N} R_i^T A_i^{-1} R_i,$$

where:

- each subdomain index i corresponds to one owned cell,
- R_i restricts to the dofs in that cell's overlap region,
- A_i is the local subdomain operator induced by that overlap region.

This is close in spirit to the older Epetra-based Camellia implementation, except that the new implementation uses Ifpack2/Tpetra infrastructure to realize the subdomains and local solves.

5.2 Recovered outer decomposition

The recovered code constructs:

- a set of overlap rows `externalGIDsThisRank`, and
- a collection of user parts `partsLocalOrdinals`, one part per owned cell,

via `getAdditiveSchwarzDetails(...)`.

These parts are then supplied to the outer Ifpack2 Schwarz layer. Thus the outer operator is effectively informed that the rank-local overlapped problem should be decomposed according to the Camellia cell-centered overlap regions.

5.3 Incorrect nested structure that was removed

Before the fix, the same overlapping user partition was also passed into the inner Block Relaxation preconditioner. In effect, this meant that the local solve inside each Schwarz application was not simply acting as a local solver for the Schwarz-local problem; it was itself being repartitioned using the same overlapping block structure.

That changed the mathematical character of the smoother from a straightforward cell-centered overlapping Schwarz application into a more complicated nested operator. Empirically, this manifested as:

- practical symmetry/SPD failures in higher-level problem tests,
- weaker or less predictable convergence behavior,
- significantly worse iteration counts than the recovered version.

5.4 Recovered structure after the fix

After the fix, the outer Schwarz layer still receives the custom user partition, but the inner Block Relaxation layer is left unpartitioned. Conceptually, this is much closer to the intended local-subdomain-solve structure.

The resulting smoother is not merely "less broken"; it is also much stronger. The iteration counts in the problem-level tests decreased dramatically, indicating that the outer Schwarz decomposition is now being used much more effectively.

6 Specific GMGOperator Changes

The most important implementation changes in `src/Solution/GMGOperator.cpp` were the following.

6.1 1. Introduction of additive Schwarz helper logic

A helper path was added in anonymous-namespace code to compute the Camellia-specific additive Schwarz data:

- overlap cells per owned cell,
- global dofs in each overlap region,
- explicit off-rank overlap GIDs,
- local overlap-map ordinals for each cell-centered Schwarz part,
- local incidence counts over the overlap map.

This logic is encapsulated in `getAdditiveSchwarzDetails(...)`.

6.2 2. Tpetra/Ifpack2 smoother construction

The smoother setup for `CAMELLIA_ADDITIVE_SCHWARZ` now uses stock Ifpack2 construction:

- create `Ifpack2::AdditiveSchwarz` through the Ifpack2 factory,
- provide `"schwarz: overlap elements"`,
- provide outer `"partitioner: type" = "user"`,
- provide outer `"partitioner: parts" = partsLocalOrdinals`.

This is the central Ifpack2-native replacement for the older bespoke Camellia additive Schwarz implementation.

6.3 3. Removal of duplicated inner user partitioner

This was a key diagnostic and implementation result, but not by itself the end of the story.

The branch established that the overlapping user partition should *not* be passed into the inner preconditioner a second time. That is, the code intentionally does *not* repeat:

- `"partitioner: type" = "user"`
- `"partitioner: parts" = partsLocalOrdinals`

inside the inner preconditioner parameter list.

This change clearly improved the behavior of the problem-level additive Schwarz preconditioning tests and removed one important source of pathological behavior. However, later multirank `GMGSolver` identity tests showed that the remaining choice of inner solver still matters. In particular, even without the duplicated inner partitioning, `Block Relaxation` still exhibits at least one small multirank failure mode inside Ifpack2 initialization.

So the correct conclusion is narrower and more precise:

> The outer overlapping per-cell user partition belongs in the Schwarz layer, not inside the inner preconditioner a second time.

That conclusion remains solid, even though the final production inner-solver policy is still unresolved.

6.4 4. Port of diagonal multiplicity weighting

The old unconditional "unimplemented" path for Camellia Schwarz diagonal weighting was replaced with a Tpetra-side multiplicity construction.

The implementation now:

1. computes local incidence counts on the overlap map,
2. records the exact overlap-map global-ordinal ordering,
3. assembles multiplicities back to the owned row map via `Epetra_FEVector`,
4. constructs $_smootherDiagonalWeight$ by setting

$$w_j = \frac{1}{m_j},$$

where m_j is the assembled multiplicity of owned dof j.

This preserves the basic Camellia idea that rows shared by multiple Schwarz domains should be weighted by the reciprocal of their multiplicity.

6.5 5. Preservation of scaling-weight logic

The existing branch logic for Schwarz scaling weight was preserved:

$$_smootherWeight = \frac{1}{1 + N_{\max}},$$

where $N_{\max}$ is based on the maximum count of side-neighbor cells in an overlap region.

This was left in place rather than reverting to the older Epetra-era scaling formula, because the goal was to stay close to the committed branch design unless forced otherwise.

7 Operator-Level Interpretation of ApplySmoother

When diagonal weighting is active, `ApplySmoother(...)` now applies the smoother in one of two weighted forms:

- if weighting is on the right, it computes a weighted residual first and then applies the smoother,
- if weighting is on the left, it applies the smoother first and then multiplies by the diagonal weight vector.

Schematically, if D is the diagonal multiplicity-weight matrix and S is the underlying Ifpack2 Schwarz smoother operator, then the weighted variants are approximately:

$$S_{\text{right}} \approx SD, \qquad S_{\text{left}} \approx DS.$$

The GMG operator uses these weighted variants in a way intended to preserve reasonable symmetry behavior when necessary, but the practical recovery result came primarily from correcting the smoother construction itself rather than from redesigning the whole V-cycle composition.

8 What the Diagnostic Phase Revealed

Several targeted diagnostics were temporarily added to `PreconditioningTests.cpp`. These were useful during recovery, even though many were later cleaned up or softened.

The most important conclusions from that phase were:

- the outer Ifpack2 overlap+user-partitioner mechanism itself works in serial and in 2-rank MPI,
- the original leading suspicion that overlapping user parts were fundamentally unsupported was not borne out,
- the practical issue was instead the duplicated use of the same partition inside the inner Block Relaxation preconditioner.

This is significant because it means the committed branch direction was fundamentally viable; it merely needed a mathematically important correction in how the smoother was assembled.

9 Symmetry and Positive-Definiteness as First-Class Recovery Signals

An important aspect of this recovery effort is that symmetry and positive-definiteness became much more explicit debugging signals than they had been in the older Ifpack-based path. During the work, practical failures appeared in the form of non-SPD behavior reported by Belos CG, and targeted diagnostics were added specifically to probe operator asymmetry and non-positive quadratic forms.

This is directly reflected in the weighted smoother-application logic in `GMGOperator`: when diagonal multiplicity weighting is active, the implementation distinguishes between left-weighted and right-weighted smoother application and uses these variants in a way intended to preserve or improve symmetry behavior. In other words, part of the recovery work was not merely to make the Ifpack2 path "run," but to make its weighted application mathematically better behaved.

An important historical implication follows. In the targeted problem-level tests, the recovered Ifpack2 path ultimately produced better practical additive Schwarz behavior than the older historical Ifpack-based baselines. This suggests that the older path may also have had latent symmetry defects, or at least symmetry-sensitive regimes that had not previously been documented explicitly. The present work therefore did more than port functionality: it surfaced symmetry/SPD sensitivity as a first-class issue and, in the tested cases, appears to have improved practical behavior relative to what had historically been accepted.

10 Effect on Problem-Level Tests

At one stage of the recovery, removing the duplicated inner user partitioner caused the additive Schwarz smoother to become dramatically stronger in the problem-level preconditioning tests, to the point that historical expected iteration counts were no longer representative.

However, subsequent broader testing showed that this behavior was not yet the final stable global outcome for the branch. Later experiments involving the inner-solver choice changed the observed iteration counts again, and some `GMGSolver` multirank identity tests still failed or crashed depending on that choice.

Therefore, the correct interpretation is:

- the duplicated inner user partitioner was indeed harmful and its removal was an important improvement,
- but the final additive Schwarz performance and robustness profile is still sensitive to the remaining inner-solver choice,
- so problem-level iteration-count baselines should not yet be treated as fully settled branch-wide truth.

11 Effect on Custom MPI Tests

The focused custom additive Schwarz tests continued to pass on 2 MPI ranks after the fix. This is important because it shows that the improved problem-level behavior did not come at the cost of breaking the low-level overlap/partition semantics in the isolated test harness.

12 Unresolved Inner-Solver Split

Later broader testing showed that the branch still has an unresolved split between two desired properties:

- `Block Relaxation` as the inner Schwarz solver, without duplicated inner user partitioning, gives the best observed behavior in the targeted problem-level additive Schwarz preconditioning tests, but
- the same `Block Relaxation` configuration still crashes in at least one small multirank `GMGSolver` identity case inside `Ifpack2::BlockRelaxation::ExtractSubmatricesStructure()`.

The diagnostic added during this phase showed that the failing `GMGSolver_UniformIdentity_1D_Slow` case is not explained by malformed parts. On the failing rank, the local structure is instead a tiny one-part, non-overlapping case with valid ordinals and incidence counts equal to one. In other words, the remaining failure appears to be a fragile `Block Relaxation` regime for a degenerate local Schwarz problem, not an overlap-membership bug or an obviously invalid user partition.

The strongest current result goes one step further: even after bypassing the custom outer Schwarz partitioning in that narrow degenerate case, and even when `ILUT` is explicitly requested as the inner preconditioner, the failing 2-rank run still routes into `Ifpack2::BlockRelaxation::ExtractSubmatrices`. This suggests that the remaining failure is now best understood as an internal Ifpack2 Schwarz/BlockRelaxation behavior in a degenerate local regime, rather than as a remaining Camellia-side outer-partitioning bug.

A trial substitution of `ILUT` as a global inner-solver replacement also significantly degraded the problem-level additive Schwarz preconditioning behavior. Thus neither "always Block Relaxation" nor "always ILUT" can yet be considered the final branch-wide answer.

This means the remaining problem is no longer the outer overlap/partition semantics, but rather the robust selection of the inner solver for all regimes of interest. It is also important to note that the exact boundary of additional multirank `GMGSolver` failures has not yet been fully mapped; at present we know that `UniformIdentity_1D_Slow` fails on 2 MPI ranks, while `UniformIdentity_2D_Slow` passes on 2 MPI ranks.

13 Files Most Materially Changed

The main files changed for the recovery were:

- `src/Solution/GMGOperator.cpp`
- `unit_tests/PreconditioningTests.cpp`

The key conceptual change lives in `GMGOperator.cpp`; the test updates mainly document and validate the recovered behavior.

14 What Was Not Revived

The uncommitted `Camellia_AdditiveSchwarz_decl.hpp` and `Camellia_AdditiveSchwarz_def.hpp` prototype was not used as the primary implementation path. It remained useful only as evidence of an earlier exploratory direction.

Similarly, the older Epetra `AdditiveSchwarz.h` implementation was used mainly as a semantic reference for overlap-domain meaning and multiplicity weighting, rather than as code to be ported wholesale.

15 Remaining Caveats

A few caveats remain:

- the final production inner-solver policy for `CAMELLIA_ADDITIVE_SCHWARZ` is still unresolved across the full test space,
- `Block Relaxation` remains the best observed choice for the targeted problem-level additive Schwarz preconditioning tests, but still crashes in at least one small multirank `GMGSolver` identity case,
- a trial substitution of `ILUT` avoided that specific crash but significantly weakened the problem-level additive Schwarz behavior,
- some diagnostic helpers added during debugging may still be worth either cleaning up further or preserving more intentionally as future debugging utilities,
- `Tpetra_Operator_to_Tpetra_Matrix::constructInverseMatrix(...)` remains effectively a stub and is not yet a full operator-materialization tool,
- `GMGSolverTests.cpp` still contains disabled SPD coverage for `CAMELLIA_ADDITIVE_SCHWARZ`, which should remain a follow-up until the inner-solver issue is stabilized.

So the remaining work is not just polish; there is still a substantive robustness issue around the inner-solver choice in the multirank GMG identity regime.

16 Bottom Line

The Ifpack2/Tpetra-based Camellia additive Schwarz recovery made real and important progress by staying close to the committed branch direction and correcting one key structural mistake:

The outer Schwarz layer should own the custom Camellia overlapping per-cell partition; the inner preconditioner should not be given that same overlapping partition a second time.

That conclusion is supported by:

- passing focused custom MPI tests,
- passing direct overlap-membership validation for the tested case,
- and improved problem-level additive Schwarz behavior under the corrected outer/inner partition separation.

However, the recovery is not yet fully complete. The robust production inner-solver choice remains unresolved across the broader multirank `GMGSolver` identity test space, so further stabilization is still required before the additive Schwarz migration should be considered fully settled.